

GCP DevOps Interview Handbook

Complete Edition – Architecture, Code & Test Cases

Prepared for: Yamuna Bandla – GCP DevOps Engineer (4+ Years)

Covers: architecture diagrams, hands-on provisioning scenarios with sample code (Terraform, Docker, Kubernetes, Jenkins, Bitbucket Pipelines), sample test cases for each scenario, concept Q&A for every resume skill, behavioral questions, troubleshooting scenarios, and a command cheat sheet.

Table of Contents

Part 1 - Orientation

1	How to Use This Handbook
2	Walking Through Your Resume – Talking Points
3	Architecture Diagrams – The Big Picture

Part 2 - Hands-On Scenarios (Code + Test Cases)

4	Scenario 1: Provisioning a GCP Network with Terraform
5	Scenario 2: IAM & Service Accounts with Terraform
6	Scenario 3: Containerizing an Application with Docker
7	Scenario 4: Deploying to Kubernetes (GKE)
8	Scenario 5: CI/CD Pipeline with Jenkins
9	Scenario 6: CI/CD Pipeline with Bitbucket Pipelines
10	Scenario 7: Monitoring & Alerting Setup
11	Scenario 8: Shell Scripting for Automation

Part 3 - Concept Reference (Q&A)

12	GCP Fundamentals
13	Docker
14	Kubernetes & GKE
15	Terraform & Infrastructure as Code
16	CI/CD – Jenkins, Bitbucket, Maven
17	Monitoring & Logging
18	Git & Version Control
19	Linux & Shell Scripting
20	Agile, Scrum & JIRA

Part 4 - Final Prep

21	Common HR / Behavioral Questions
22	Scenario-Based Troubleshooting Questions
23	Quick Command Cheat Sheet
24	Final Tips for Interview Day

1. How to Use This Handbook

This edition goes deeper than a plain Q&A sheet. Part 2 walks through eight real provisioning/configuration scenarios – the kind an interviewer might ask you to whiteboard or explain step-by-step – each with an architecture diagram, sample code you can actually type from memory (or paraphrase), and sample test cases showing how you'd verify the work. Part 3 is the concept reference from the earlier version, kept for quick review. Part 4 is behavioral prep, troubleshooting scenarios, and a command cheat sheet.

Suggested study order: Chapter 2 (your story) → Chapter 3 (the big picture diagrams) → Part 2 scenarios in order, since each builds on the last (network → identity → container → Kubernetes → pipeline → monitoring → scripting) → Part 3 as reinforcement → Part 4 the night before.

Tip: You don't need to memorize every line of code. Interviewers care that you understand *why* each piece exists and can talk through it. Read each code sample, then close the handbook and try to explain what it does line-by-line out loud.

2. Walking Through Your Resume - Talking Points

Interviewers almost always start with "walk me through your resume" or "tell me about a recent project." Here is how to talk about each part of your experience in plain language, matched to the bullet points on your resume.

Overall Summary (30-second version)

"I'm a DevOps engineer with about 4 years of experience working on Google Cloud Platform. Day to day, I work on setting up cloud infrastructure using Terraform, building CI/CD pipelines with Jenkins and Bitbucket, and supporting the deployment of applications – including some containerization with Docker and deployments to Kubernetes on GKE. I also keep an eye on system health using GCP's monitoring and logging tools, and I work in Agile teams tracking work through JIRA."

Project 1: Lumen Technologies (Sep 2024 - Present)

"Supported migration of backend services to a modernized architecture on GCP..."

How to explain it: "We were moving some of our backend services to a newer setup on GCP. My part was making sure the new environment matched what the old one needed – checking configurations, testing things after the move, and helping catch issues early so there wasn't downtime for users."

"Provisioned and maintained core GCP infrastructure using Terraform..."

How to explain it: "Instead of manually creating networks and firewall rules in the GCP console, we wrote Terraform scripts so the same environment could be recreated reliably. I worked on the VPC, subnet, and firewall rule definitions, and ran `terraform plan` and `apply` as part of our setup process." (See Chapter 4 for the full working example.)

"Built and maintained CI/CD pipelines using Jenkins and Maven..."

How to explain it: "I worked on our Jenkins pipelines that build the code with Maven, run basic checks, and then deploy it. This cut down on manual steps." (See Chapter 8 for a sample Jenkinsfile.)

"Containerized application components using Docker and assisted with deployments to GKE..."

How to explain it: "For a couple of components, I wrote Dockerfiles to package them up, and helped with getting those containers running on our GKE cluster." (See Chapters 6 and 7 for sample Dockerfiles and manifests.)

"Monitored system health using GCP Cloud Monitoring and Cloud Logging..."

How to explain it: "When something went wrong in production, I'd check Cloud Logging first to find the error, then use Cloud Monitoring dashboards to see if it was a resource issue." (See Chapter 10.)

Project 2: Hyperoptics - British Telecommunication Company (May 2022 - Aug 2024)

"Deployed and configured 50+ backend services on GCP..."

How to explain it: "This was a larger project with many services. My role was setting these up on GCP consistently – same patterns for networking and configuration – so the team wasn't reinventing the setup each time."

"Automated deployments using Bitbucket Pipelines and Maven, reducing deployment time by ~30%..."

How to explain it: "Before automation, deployments were partly manual. I helped set up Bitbucket Pipelines so builds and deployments triggered automatically on code merges." (See Chapter 9.)

"Used Terraform to provision GCP infrastructure consistently across environments..."

How to explain it: "We had multiple environments – dev, test, prod – and Terraform let us apply the same infrastructure definitions to each one with small variable changes."

"Implemented basic security measures including OAuth 2.0 and CORS configuration..."

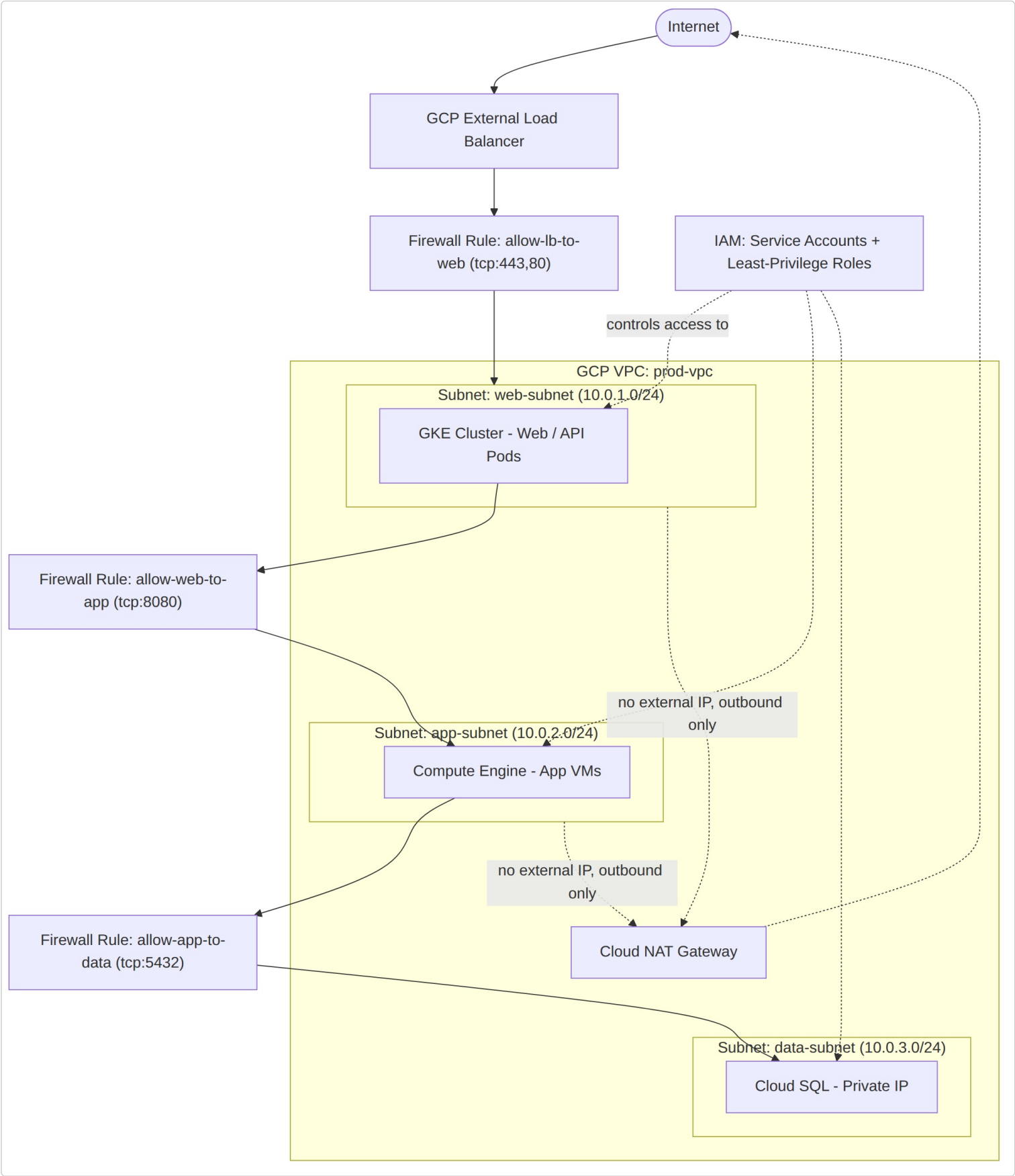
How to explain it: "Since this was for a telecom client, security mattered. I worked on making sure services validated tokens properly with OAuth 2.0, and configured CORS so only approved origins could call our APIs."

Tip: If asked "what was the hardest part of that project," pick one bullet above and describe a specific problem, what you tried, and what fixed it – a short story is more memorable than a list of tools.

3. Architecture Diagrams - The Big Picture

Before diving into individual scenarios, it helps to see how everything fits together. These four diagrams cover the network layout, the end-to-end CI/CD flow, how a Kubernetes deployment is structured, and the Terraform workflow. Interviewers often ask you to "draw the architecture" - practice sketching simplified versions of these on a whiteboard or notepad.

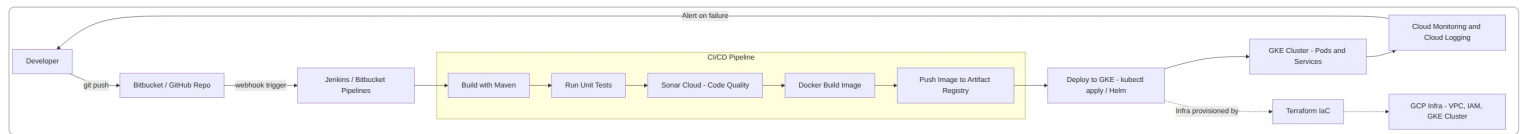
3.1 GCP Network Architecture



Public traffic hits the load balancer, which is the only thing allowed through the firewall from the internet. Web, app, and data tiers sit in separate subnets, each with its own firewall rule controlling what can talk to it. VMs and Pods have no external IP - Cloud NAT lets them reach the internet outbound (e.g., to download packages) without being directly reachable from it.

How to talk about it: "We use a single VPC with subnets separated by tier - web, app, and data. Firewall rules are scoped narrowly, so the web tier can only be reached from the load balancer, and the data tier can only be reached from the app tier. Nothing in app or data has a public IP; outbound internet access goes through Cloud NAT. IAM controls which service accounts and users can touch which resources."

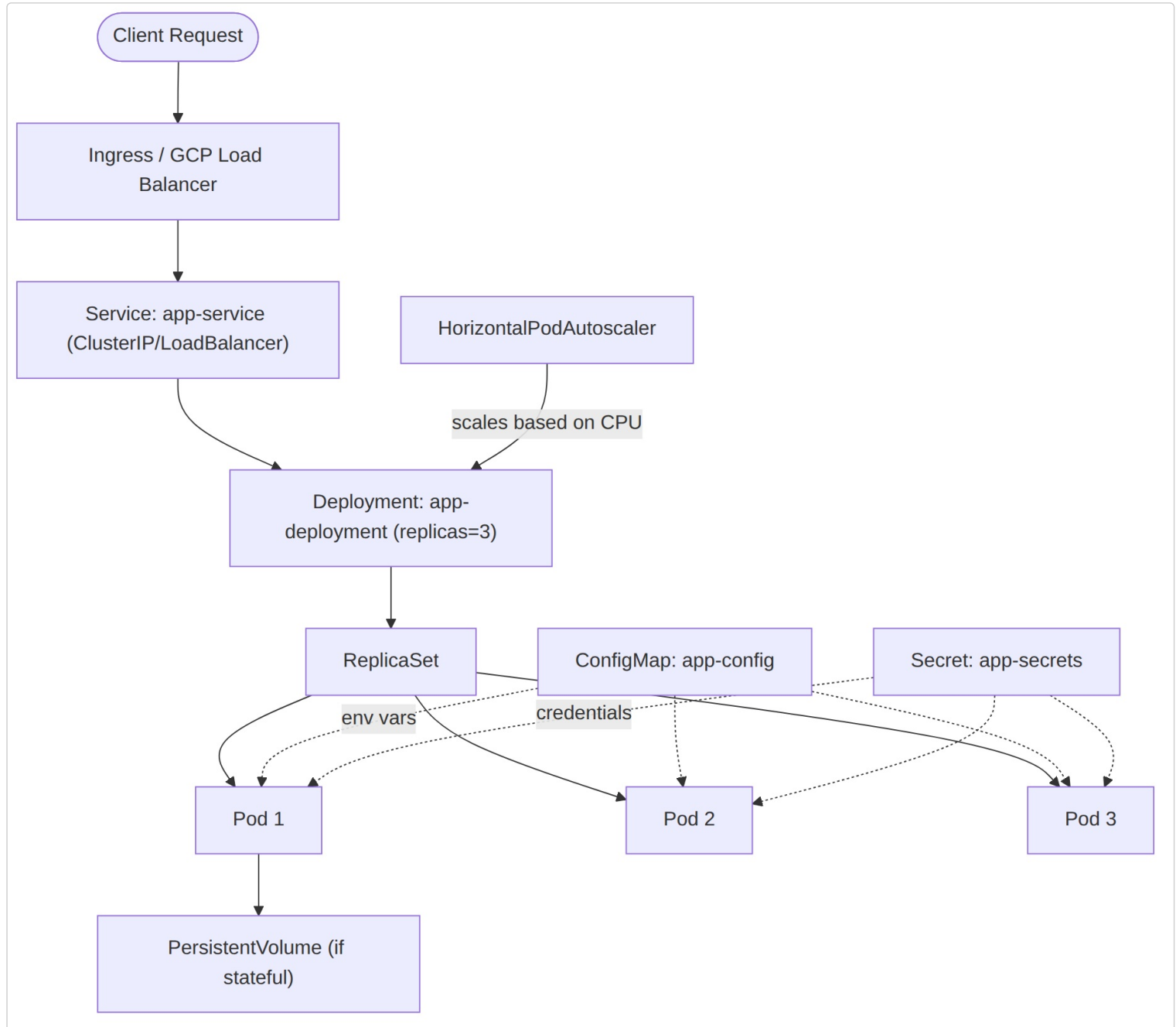
3.2 End-to-End CI/CD Architecture



A commit triggers the pipeline: build with Maven, run tests, check code quality with Sonar Cloud, build a Docker image, push it to Artifact Registry, then deploy to GKE. Cloud Monitoring watches the result and can alert the team if something goes wrong. The underlying infrastructure (VPC, IAM, the GKE cluster itself) is provisioned separately by Terraform.

How to talk about it: "A developer pushes code, which triggers Jenkins or Bitbucket Pipelines. The pipeline builds and tests the code, runs a Sonar Cloud quality check, builds a Docker image, and pushes it to Artifact Registry. Then it deploys the new image to our GKE cluster. Once it's live, Cloud Monitoring and Logging keep watch, and we get alerted if error rates spike."

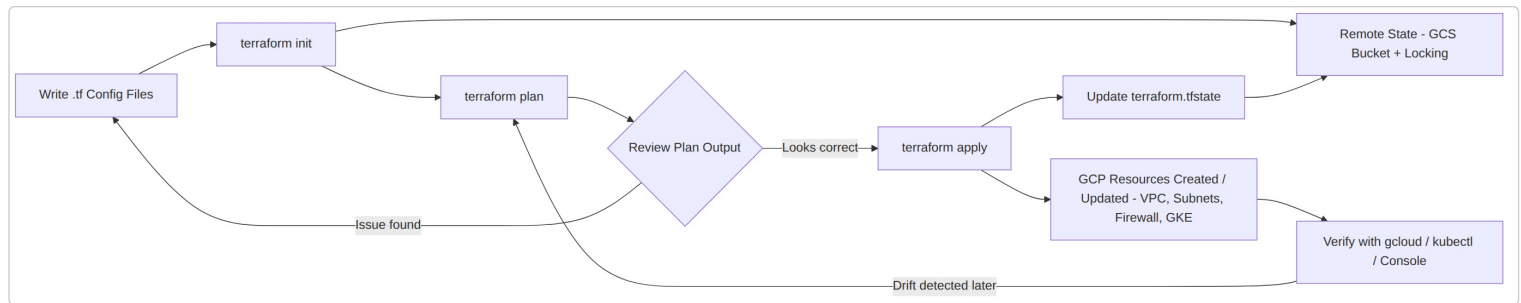
3.3 Kubernetes Deployment Architecture



Client traffic goes through an Ingress/Load Balancer to a Service, which routes to Pods managed by a Deployment (via a ReplicaSet). ConfigMaps and Secrets inject configuration and credentials into each Pod without hardcoding them in the image. A HorizontalPodAutoscaler adjusts the number of Pods based on load.

How to talk about it: "Traffic comes in through a load balancer to a Service, which spreads requests across however many Pods the Deployment is currently running. Configuration and secrets are injected through ConfigMaps and Secrets rather than baked into the image, so we can change them without rebuilding. If load increases, the HorizontalPodAutoscaler adds more Pods automatically."

3.4 Terraform Workflow



The standard loop: write configuration, initialize, plan (preview), review, apply. State is stored remotely (a GCS bucket) so the team shares one source of truth with locking to prevent conflicting changes. If someone changes something manually later, the next plan will show that drift.

How to talk about it: "We keep our Terraform state in a remote GCS bucket with locking enabled, so the whole team works off the same state safely. The workflow is always init, plan, review, then apply – we never apply without reviewing the plan output first, especially in production."

4. Scenario 1: Provisioning a GCP Network with Terraform

Interview framing: "Walk me through how you'd set up the networking for a new GCP project." or "Write Terraform to create a VPC with a couple of subnets and a firewall rule."

Refer back to the network diagram in Chapter 3.1. The goal: one VPC, tiered subnets, firewall rules scoped per-tier, and Cloud NAT so private resources can still reach the internet outbound.

main.tf

```
resource "google_compute_network" "prod_vpc" {
  name          = "prod-vpc"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "web_subnet" {
  name          = "web-subnet"
  ip_cidr_range = "10.0.1.0/24"
  region        = "us-central1"
  network       = google_compute_network.prod_vpc.id
}

resource "google_compute_subnetwork" "app_subnet" {
  name          = "app-subnet"
  ip_cidr_range = "10.0.2.0/24"
  region        = "us-central1"
  network       = google_compute_network.prod_vpc.id
  private_ip_google_access = true
}

# Allow the load balancer / internet to reach the web tier only, on 80/443
resource "google_compute_firewall" "allow_lb_to_web" {
  name          = "allow-lb-to-web"
  network       = google_compute_network.prod_vpc.id
  direction     = "INGRESS"
  allow {
    protocol = "tcp"
    ports    = ["80", "443"]
  }
  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["web"]
}

# Allow the web tier to reach the app tier only, on 8080
resource "google_compute_firewall" "allow_web_to_app" {
  name          = "allow-web-to-app"
  network       = google_compute_network.prod_vpc.id
  direction     = "INGRESS"
  allow {
    protocol = "tcp"
    ports    = ["8080"]
  }
  source_tags   = ["web"]
  target_tags   = ["app"]
}

# Cloud NAT so private subnets can reach the internet outbound (e.g. for OS updates)
resource "google_compute_router" "nat_router" {
  name          = "nat-router"
  network       = google_compute_network.prod_vpc.id
  region        = "us-central1"
}

resource "google_compute_router_nat" "nat_gateway" {
  name          = "nat-gateway"
  router        = google_compute_router.nat_router.name
  region        = "us-central1"
  nat_ip_allocate_option = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"
}
```

backend.tf – remote state

```
terraform {
  backend "gcs" {
    bucket = "my-project-tf-state"
    prefix = "network"
  }
}
```

Sample Test Cases

ID	Description	Steps	Expected Result
TC-NET-01	VPC is created with custom subnet mode	<code>terraform apply</code> , then <code>gcloud compute networks describe prod-vpc</code>	Network exists; <code>autoCreateSubnetworks: false</code>
TC-NET-02	Subnet CIDR ranges don't overlap	<code>terraform plan</code> after adding a new subnet	Plan succeeds with no CIDR conflict error
TC-NET-03	Firewall allows only 80/443 from the internet to web tier	<code>gcloud compute firewall-rules describe allow-lb-to-web</code>	Ports match exactly 80,443; source range 0.0.0.0/0 only on this rule
TC-NET-04	App-tier VM has no external IP	<code>gcloud compute instances describe <vm> --format="get(networkInterfaces[0].accessConfigs) "</code>	Empty / no access config (confirms no public IP)
TC-NET-05	Outbound internet access still works via NAT	SSH into an app-tier VM (via IAP) and run <code>curl -I https://google.com</code>	Returns HTTP 200/301, confirming NAT egress works
TC-NET-06 (negative)	Internet cannot reach the app tier directly	From an external machine, attempt <code>curl http://<app-vm-internal-ip>:8080</code>	Connection times out / refused – no firewall rule permits it

Talking point: Explain *why* you separated firewall rules per tier instead of one broad rule – it limits the blast radius if one tier is compromised (defense in depth).

5. Scenario 2: IAM & Service Accounts with Terraform

Interview framing: "How would you set up access for a CI/CD pipeline to deploy to GKE without giving it too much access?"

iam.tf

```
resource "google_service_account" "ci_deployer" {
  account_id = "ci-deployer"
  display_name = "CI/CD Deployer Service Account"
}

# Least-privilege: only what's needed to deploy to GKE, nothing else
resource "google_project_iam_member" "ci_deployer_gke_role" {
  project = var.project_id
  role    = "roles/container.developer"
  member  = "serviceAccount:${google_service_account.ci_deployer.email}"
}

resource "google_project_iam_member" "ci_deployer_artifact_registry" {
  project = var.project_id
  role    = "roles/artifactregistry.writer"
  member  = "serviceAccount:${google_service_account.ci_deployer.email}"
}

resource "google_service_account_key" "ci_deployer_key" {
  service_account_id = google_service_account.ci_deployer.name
}
```

Sample Test Cases

ID	Description	Steps	Expected Result
TC-IAM-01	Service account has only the intended roles	<code>gcloud projects get-iam-policy PROJECT --flatten="bindings[].members" --filter="bindings.members:ci-deployer"</code>	Only <code>container.developer</code> and <code>artifactregistry.writer</code> listed
TC-IAM-02	Pipeline can deploy using the service account	Run a pipeline deploy step authenticated as <code>ci-deployer</code>	Deployment to GKE succeeds
TC-IAM-03 (negative)	Service account cannot perform unrelated destructive actions	Authenticated as <code>ci-deployer</code> , attempt <code>gcloud compute networks delete prod-vpc</code>	Permission denied (403) - confirms least privilege is enforced
TC-IAM-04 (negative)	Key rotation doesn't break running pipelines mid-flight	Rotate the service account key, re-run pipeline with new key	Old key is invalidated; new key authenticates successfully

Talking point: Be ready to explain the difference between a broad role like `roles/editor` and narrow roles like `roles/container.developer` - this is one of the most common IAM interview questions.

6. Scenario 3: Containerizing an Application with Docker

Interview framing: "Write a Dockerfile for a Java/Maven application and tell me how you'd keep the image small and secure."

Dockerfile (multi-stage build)

```
# Stage 1: Build the application with Maven
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run it on a small, minimal runtime image
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=build /app/target/*.jar app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why it's written this way: the multi-stage build keeps Maven and all the build tools out of the final image – only the compiled JAR ships. The Alpine-based JRE image is much smaller than a full JDK image. Running as a non-root `appuser` instead of root reduces the impact if the container is ever compromised.

.dockerignore

```
target/
.git/
*.md
.env
```

Sample Test Script – Build, Run, Health-Check

test_docker.sh

```
#!/bin/bash
set -e
docker build -t myapp:test .
docker run -d -p 8080:8080 --name myapp-test myapp:test
sleep 5

STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/health)
if [ "$STATUS" -eq 200 ]; then
    echo "PASS: Health check returned 200"
else
    echo "FAIL: Health check returned $STATUS"
    docker logs myapp-test
    docker stop myapp-test && docker rm myapp-test
    exit 1
fi

docker stop myapp-test && docker rm myapp-test
```

Sample Test Cases

ID	Description	Steps	Expected Result
TC-DOCK-01	Image builds successfully	<code>docker build -t myapp:test .</code>	Build completes with exit code 0
TC-DOCK-02	Final image size is reasonable	<code>docker images myapp:test</code>	Image size well under a full JDK-based image (Alpine JRE base keeps it small)
TC-DOCK-03	Container starts and app responds	Run <code>test_docker.sh</code>	Health endpoint returns HTTP 200
TC-DOCK-04	Container runs as non-root	<code>docker exec myapp-test whoami</code>	Returns <code>appuser</code> , not <code>root</code>
TC-DOCK-05 (negative)	Missing required env var fails clearly, not silently	Run container without a required <code>DB_HOST</code> env var	Container exits with a clear error in logs, not a hanging/unclear failure
TC-DOCK-06 (negative)	Invalid image tag fails to pull	<code>docker run myapp:doesnotexist</code>	Clear "manifest not found" error

7. Scenario 4: Deploying to Kubernetes (GKE)

Interview framing: "How would you deploy this container to GKE with basic resilience - multiple replicas, config separated from code, and auto-scaling?"

Refer to the Kubernetes diagram in Chapter 3.3. This ties four objects together: a Deployment (runs the Pods), a Service (stable network entry point), a ConfigMap/Secret (external configuration), and an HPA (auto-scaling).

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: us-central1-docker.pkg.dev/my-project/my-repo/myapp:1.0.0
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef: { name: app-config }
            - secretRef: { name: app-secrets }
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 15
          resources:
            requests: { cpu: "250m", memory: "256Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: app-service
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 8080
```

configmap.yaml & hpa.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  ENVIRONMENT: "production"
...
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

Sample Test Cases

ID	Description	Steps	Expected Result
TC-K8S-01	All replicas come up healthy	<code>kubectl apply -f .</code> , then <code>kubectl get pods</code>	3/3 pods show <code>Running</code> and <code>1/1 Ready</code>
TC-K8S-02	Service routes traffic to healthy pods	<code>kubectl port-forward svc/app-service 8080:80</code> , then curl it	Returns a successful response from one of the pods
TC-K8S-03	Rolling update causes no downtime	Update image tag, <code>kubectl apply</code> , hit the service continuously during rollout	No failed requests; old pods stay up until new ones pass readiness
TC-K8S-04	HPA scales up under load	Generate load (e.g., with a load-testing tool), watch <code>kubectl get hpa</code>	Replica count increases toward <code>maxReplicas</code> as CPU exceeds 70%
TC-K8S-05 (negative)	Bad image tag is caught, not silently deployed	Deploy with a non-existent tag	<code>kubectl get pods</code> shows <code>ImagePullBackOff</code> ; rollout does not proceed to replace healthy pods

TC-K8S-06 (negative)	Pod without required config fails readiness, not silently serves errors	Remove the ConfigMap reference and redeploy	Pod fails readiness probe and is not added to the Service's endpoints
----------------------	---	---	---

Talking point: Explain why readiness and liveness probes are both needed - readiness controls whether a pod *receives* traffic, liveness controls whether Kubernetes *restarts* a pod that's stuck.

8. Scenario 5: CI/CD Pipeline with Jenkins

Interview framing: "Describe a Jenkins pipeline that builds, tests, and deploys a Java application to GKE."

Jenkinsfile

```
pipeline {
  agent any
  environment {
    IMAGE = "us-central1-docker.pkg.dev/my-project/my-repo/myapp"
    TAG   = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build & Unit Test') {
      steps { sh 'mvn clean package' }
    }
    stage('Code Quality') {
      steps { sh 'mvn sonar:sonar' }
    }
    stage('Build Docker Image') {
      steps { sh 'docker build -t $IMAGE:$TAG .' }
    }
    stage('Push Image') {
      steps { sh 'docker push $IMAGE:$TAG' }
    }
    stage('Deploy to GKE') {
      steps {
        sh 'gcloud container clusters get-credentials my-cluster --zone us-central1-a'
        sh 'kubectl set image deployment/app-deployment myapp=$IMAGE:$TAG'
        sh 'kubectl rollout status deployment/app-deployment --timeout=120s'
      }
    }
  }
  post {
    failure {
      echo 'Pipeline failed - notifying team'
      // mail to: 'team@company.com', subject: "Build ${env.BUILD_NUMBER} failed"
    }
    success {
      echo 'Deployment successful'
    }
  }
}
```

Why `rollout status` matters: it makes the pipeline actually wait and confirm the new pods became healthy, instead of declaring success the moment `kubectl set image` is accepted. If the rollout times out, the pipeline stage fails and you know immediately.

Sample Test Cases

ID	Description	Steps	Expected Result
TC-CI-01	Pipeline fails fast on a failing unit test	Introduce a failing test, trigger the pipeline	Pipeline stops at "Build & Unit Test" stage; does not proceed to deploy
TC-CI-02	Successful pipeline deploys the new image	Merge a small change, let the pipeline run end-to-end	<code>kubectl get deployment app-deployment -o jsonpath='{.spec.template.spec.containers[0].image}'</code> shows the new tag
TC-CI-03 (negative)	Bad deploy is caught by rollout status, not marked "success"	Push an image tag that causes CrashLoopBackOff	"Deploy to GKE" stage fails on rollout timeout; pipeline is marked failed, not successful
TC-CI-04	Failure notification actually fires	Force a stage failure	Team is notified (email/chat) per the <code>post { failure { ... } }</code> block

9. Scenario 6: CI/CD Pipeline with Bitbucket Pipelines

Interview framing: "Your resume mentions Bitbucket Pipelines reducing deployment time by ~30% - show me roughly what that pipeline looked like."

bitbucket-pipelines.yml

```
image: maven:3.9-eclipse-temurin-17

pipelines:
  branches:
    main:
      - step:
          name: Build and Test
          caches:
            - maven
          script:
            - mvn clean package
      - step:
          name: Build and Push Docker Image
          services:
            - docker
          script:
            - docker build -t $IMAGE_NAME:$BITBUCKET_BUILD_NUMBER .
            - echo "$GCLOUD_API_KEYFILE" | docker login -u _json_key --password-stdin https://us-central1-docker.pkg.dev
            - docker push $IMAGE_NAME:$BITBUCKET_BUILD_NUMBER
      - step:
          name: Deploy to GKE
          script:
            - pipe: google-cloud/gke-kubect1-run
              variables:
                KEY_FILE: $GCP_API_KEYFILE
                PROJECT: $GCP_PROJECT
                COMPUTE_ZONE: $GCP_ZONE
                CLUSTER_NAME: $GKE_CLUSTER
                KUBECTL_COMMAND: 'set image deployment/app-deployment myapp=$IMAGE_NAME:$BITBUCKET_BUILD_NUMBER'
```

Where the ~30% time saving comes from: before this pipeline existed, someone had to manually build the JAR, build and push the Docker image, and run `kubect1` commands by hand for every release. Automating those three steps removes the manual handoffs and waiting between them.

Sample Test Cases

ID	Description	Steps	Expected Result
TC-BB-01	Pipeline triggers automatically on merge to main	Merge a pull request into <code>main</code>	Pipeline run starts within seconds, visible in Bitbucket Pipelines UI
TC-BB-02	Maven cache speeds up repeated builds	Run the pipeline twice in a row	Second run's "Build and Test" step is noticeably faster (dependencies cached)
TC-BB-03 (negative)	Docker login failure stops the pipeline before deploy	Use an expired/invalid <code>GCLOUD_API_KEYFILE</code>	"Build and Push Docker Image" step fails; "Deploy to GKE" step never runs
TC-BB-04	Deployment step targets the correct cluster	Check pipeline logs for the <code>gke-kubect1-run</code> step	Logs confirm correct project, zone, and cluster name were used

10. Scenario 7: Monitoring & Alerting Setup

Interview framing: "How would you get notified automatically if your service starts returning a lot of errors?"

monitoring.tf

```
resource "google_monitoring_notification_channel" "email" {
  display_name = "DevOps Team Email"
  type        = "email"
  labels = {
    email_address = "devops-team@company.com"
  }
}

resource "google_monitoring_alert_policy" "high_error_rate" {
  display_name = "High 5xx Error Rate"
  combiner     = "OR"

  conditions {
    display_name = "5xx errors above 5% for 5 minutes"
    condition_threshold {
      filter      = "metric.type=\"loadbalancing.googleapis.com/https/request_count\" AND metric.label.response_code_class=\"500\""
      comparison  = "COMPARISON_GT"
      threshold_value = 0.05
      duration    = "300s"
      aggregations {
        alignment_period = "60s"
        per_series_aligner = "ALIGN_RATE"
      }
    }
  }

  notification_channels = [google_monitoring_notification_channel.email.id]
}
```

How you'd describe the everyday version: "For quick day-to-day checks I'd use Cloud Logging to search for error-level logs around a specific time window, and Cloud Monitoring dashboards to see CPU, memory, and request latency. For anything that should page the team automatically, we set up an alert policy like this one so we don't have to be watching a dashboard constantly."

Sample Test Cases

ID	Description	Steps	Expected Result
TC-MON-01	Alert fires when error rate crosses the threshold	Synthetically generate 500 errors above 5% of traffic for 5+ minutes	Alert policy triggers; notification email is received within a few minutes
TC-MON-02 (negative)	Alert does not fire on a brief, small spike	Generate a short burst of errors lasting under the 5-minute duration	No alert fires (confirms duration threshold prevents noisy alerts)
TC-MON-03	Relevant logs are searchable during an incident	During the synthetic error test, search Cloud Logging by severity=ERROR	The generated errors appear with timestamps and request context

11. Scenario 8: Shell Scripting for Automation

Interview framing: "Write a script that checks whether a deployment was successful and rolls it back automatically if not."

deploy_and_verify.sh

```
#!/bin/bash
set -euo pipefail

DEPLOYMENT="app-deployment"
NAMESPACE="default"
NEW_IMAGE="$1"

echo "Deploying image: $NEW_IMAGE"
PREVIOUS_IMAGE=$(kubectl get deployment "$DEPLOYMENT" -n "$NAMESPACE" \
  -o jsonpath='{.spec.template.spec.containers[0].image}')

kubectl set image deployment/"$DEPLOYMENT" myapp="$NEW_IMAGE" -n "$NAMESPACE"

echo "Waiting for rollout to complete..."
if kubectl rollout status deployment/"$DEPLOYMENT" -n "$NAMESPACE" --timeout=120s; then
  echo "PASS: Deployment succeeded."
else
  echo "FAIL: Rollout did not complete in time. Rolling back to $PREVIOUS_IMAGE"
  kubectl set image deployment/"$DEPLOYMENT" myapp="$PREVIOUS_IMAGE" -n "$NAMESPACE"
  kubectl rollout status deployment/"$DEPLOYMENT" -n "$NAMESPACE" --timeout=120s
  echo "Rolled back successfully."
  exit 1
fi
```

Usage: ./deploy_and_verify.sh us-centrall-docker.pkg.dev/my-project/my-repo/myapp:1.2.0 . This is the kind of script that would sit inside the "Deploy to GKE" stage of the Jenkinsfile or Bitbucket pipeline from Chapters 8-9, so a bad rollout doesn't need a human to notice and manually roll back.

Sample Test Cases

ID	Description	Steps	Expected Result
TC-SH-01	Script deploys a valid image successfully	Run script with a known-good image tag	Prints "PASS", exits with code 0, deployment shows the new image
TC-SH-02 (negative)	Script rolls back on a bad image	Run script with a tag that causes CrashLoopBackOff	Prints "FAIL" and rollback message; deployment ends up back on the previous working image; script exits 1
TC-SH-03	Script fails safely on a typo'd deployment name	Run with an incorrect <code>DEPLOYMENT</code> value	Because of <code>set -e</code> , the script exits immediately on the first failing command rather than continuing silently

Talking point: Mention `set -euo pipefail` specifically if asked about writing "safe" shell scripts – it's a small detail that signals real hands-on experience: `-e` exits on error, `-u` errors on undefined variables, and `-o pipefail` catches failures inside piped commands.

12. GCP Fundamentals

Compute & Storage

Q: What is Compute Engine and when would you use it over other GCP compute options?

A: Compute Engine gives you virtual machines (VMs) that you fully manage – you choose the OS, install software, and control scaling yourself. You'd use it when you need full control over the environment. For containerized apps you'd usually prefer GKE, and for simple event-driven code you might use Cloud Functions or Cloud Run instead, since they need less management.

Q: What is Cloud Storage and how is it different from a persistent disk?

A: Cloud Storage is object storage – good for files, backups, static assets, and logs, accessed over HTTP(S) as objects in buckets. A persistent disk is block storage attached directly to a VM, like a hard drive, used for the operating system and application data that needs a filesystem.

Networking

Q: What is a VPC?

A: A Virtual Private Cloud (VPC) is an isolated, private network within GCP where you place your resources. It lets you control IP ranges, routing, and which resources can talk to each other, similar to how you'd design a private network on-premises.

Q: What is a subnet, and why do we divide a VPC into subnets?

A: A subnet is a smaller IP range carved out of a VPC, usually tied to a specific region. We divide a VPC into subnets to organize resources logically (for example, separating a web tier from a database tier, as shown in Chapter 3.1) and to control which resources are reachable from which regions.

Q: What does a firewall rule do in GCP, and how is it different from a traditional on-prem firewall?

A: A GCP firewall rule controls traffic in and out of resources based on IP range, protocol, port, and tags/service accounts. It's software-defined at the network level rather than a physical appliance, and rules apply per-VPC, so you define explicitly what's allowed in (ingress) and out (egress); by default, most inbound traffic is denied unless a rule allows it.

Q: What is Cloud NAT and why do private VMs need it?

A: Cloud NAT (Network Address Translation) lets VMs that only have private, internal IP addresses reach the internet (for example, to download OS updates) without being directly exposed to inbound internet traffic. It translates outbound requests to a shared external IP so the VM itself stays private and secure.

Q: Can you explain the difference between an external and internal load balancer at a basic level?

A: An external load balancer accepts traffic from the public internet and distributes it to backend services; an internal load balancer only distributes traffic between resources inside your VPC (for example, one internal service calling another) and isn't reachable from outside.

Identity & Access Management (IAM)

Q: What is IAM in GCP and why does it matter for a DevOps role?

A: IAM controls who (a user, group, or service account) can do what (a role, like Viewer, Editor, or a custom role) on which resource. As a DevOps engineer, you regularly create service accounts for pipelines and applications, and you need to grant only the permissions they actually need – this is the principle of least privilege (see Chapter 5 for a full example).

Q: What is a service account, and how is it different from a regular user account?

A: A service account is an identity used by an application, VM, or pipeline (not a person) to authenticate and call GCP APIs. Unlike a user account, it isn't tied to a human login – it typically authenticates using a key file or attached identity, and it's used so automated processes like Jenkins jobs or Terraform runs can act on GCP resources securely.

Q: What does "least privilege" mean, and can you give an example from your work?

A: Least privilege means giving an identity only the permissions it needs to do its job, nothing more. For example, a CI/CD service account that only needs to deploy to one project shouldn't also have permission to delete resources in every other project – you'd scope its role narrowly to reduce risk if it's ever compromised.

Certifications-Related

Q: You have the GCP Associate Cloud Engineer certification - what kind of topics does that cover?

A: It covers setting up a cloud environment, planning and configuring resources like Compute Engine, GKE, storage and networking basics, managing IAM roles, and monitoring resources – essentially the day-to-day operational skills for running workloads on GCP, which lines up closely with the DevOps work I do.

Q: You're also a HashiCorp Certified Terraform Associate - what does that certification validate?

A: It validates understanding of Infrastructure as Code concepts, Terraform's core workflow (write, plan, apply), state management, variables and outputs, modules, and provisioning resources against a cloud provider – the fundamentals needed to manage infrastructure safely with Terraform.

13. Docker

Q: What is Docker and what problem does it solve?

A: Docker packages an application with everything it needs to run (code, libraries, runtime, settings) into a container. This solves the "it works on my machine" problem, because the container runs the same way in development, testing, and production.

Q: What is the difference between an image and a container?

A: An image is a read-only template – the packaged application and its dependencies. A container is a running instance of that image. You can start many containers from the same image, similar to how a class and an object relate to each other in programming.

Q: What is a Dockerfile?

A: A Dockerfile is a text file with step-by-step instructions for building an image (see the working example in Chapter 6).

Q: What's the difference between `CMD` and `ENTRYPOINT` in a Dockerfile?

A: Both define what runs when the container starts. `ENTRYPOINT` sets the main command that's harder to override, while `CMD` provides default arguments that are easier to override at runtime.

Q: How would you reduce the size of a Docker image?

A: Use a smaller base image (like an "alpine" variant), avoid installing unnecessary packages, combine commands to reduce layers, and use multi-stage builds so build tools aren't included in the final image – all of which are demonstrated in the Chapter 6 Dockerfile.

Q: What is a Docker volume and why would you use one?

A: A volume is a way to persist data outside the container's own filesystem. Since containers are meant to be disposable, anything written inside the container is lost when it's removed – a volume lets data like database files survive container restarts or replacements.

Q: How do containers on the same host talk to each other?

A: Docker creates virtual networks that containers can join. Containers on the same custom network can reach each other by container name; Docker also supports port mapping so a container's internal port is exposed on the host machine.

14. Kubernetes & GKE

Q: What problem does Kubernetes solve that Docker alone doesn't?

A: Docker runs individual containers, but doesn't manage many containers across multiple machines, restart them if they crash, or scale them automatically. Kubernetes orchestrates containers across a cluster – scheduling them, restarting failed ones, scaling them, and managing networking between them.

Q: What is GKE?

A: Google Kubernetes Engine (GKE) is Google's managed Kubernetes service. Google manages the control plane (the "brain" of the cluster) for you, so you mainly focus on your worker nodes and the workloads you deploy, rather than setting up Kubernetes from scratch.

Q: What is a Pod?

A: A Pod is the smallest deployable unit in Kubernetes – it wraps one or more tightly coupled containers that share networking and storage. Most of the time, a Pod runs a single application container.

Q: What is a Deployment, and why not just create Pods directly?

A: A Deployment manages a set of identical Pods and keeps the desired number running – if a Pod crashes, the Deployment creates a replacement automatically. It also manages rolling updates (see Chapter 7).

Q: What is a Service in Kubernetes?

A: A Service gives a stable network address (a name and IP) to a group of Pods, even though individual Pods can be replaced and get new IPs over time.

Q: What's the difference between a ClusterIP, NodePort, and LoadBalancer Service?

A: ClusterIP is only reachable inside the cluster (the default). NodePort opens a specific port on every node so it's reachable from outside on that port. LoadBalancer provisions an external load balancer (in GKE, a GCP load balancer) to expose the Service to the internet.

Q: What is a namespace used for?

A: A namespace is a way to logically separate resources within the same cluster – for example, separating "dev," "test," and "prod" workloads, or separating teams.

Q: How would you check why a Pod is failing?

A: Start with `kubectl get pods` to see its status, then `kubectl describe pod <name>` to see events, and `kubectl logs <name>` to see application output for errors.

Be honest about depth: Your resume lists GKE as "working knowledge," so it's fine to say "I've deployed and troubleshot workloads on an existing GKE cluster, but I haven't set up a cluster from scratch" if asked directly.

15. Terraform & Infrastructure as Code

Q: What is Infrastructure as Code, and why is it useful?

A: IaC means defining your infrastructure in text/configuration files instead of clicking through a console. It's repeatable, version-controlled, reviewable, and lets you recreate the same environment reliably across dev, test, and prod (see Chapters 4–5 for full examples).

Q: What are the main Terraform commands you use, in order?

A: `terraform init` sets up the working directory and downloads providers; `terraform plan` shows what changes would be made without applying them; `terraform apply` actually creates/updates/deletes resources; `terraform destroy` tears down the resources it manages.

Q: What is Terraform state, and why does it matter?

A: State is a file that tracks what resources Terraform has created and their current configuration. Terraform uses it to know what already exists, so it can figure out the difference between your desired configuration and reality.

Q: Why would you use remote state instead of a local file?

A: Remote state (like the GCS backend in Chapter 4) lets a team share the same state safely, supports locking so two people don't apply changes at the same time and corrupt the state, and keeps the state backed up.

Q: What is a Terraform module?

A: A module is a reusable, self-contained group of Terraform configuration – for example, a module that creates a standard VPC with subnets and firewall rules, instead of rewriting that configuration for every project.

Q: What's the difference between a variable and an output in Terraform?

A: A variable is an input you pass into your configuration so the same code can be reused with different values. An output is a value your configuration exposes after applying, for other configurations or team members to reference.

Q: What happens if someone manually changes a resource in the GCP console that Terraform manages?

A: The next `terraform plan` will detect "drift" and show that it wants to change the resource back to match the configuration (or you'd need to import the change intentionally).

16. CI/CD - Jenkins, Bitbucket, Maven

Q: What is CI/CD, in your own words?

A: Continuous Integration means regularly merging code changes and automatically building/testing them so issues are caught early. Continuous Delivery/Deployment means automating the steps to get that tested code out to an environment reliably and repeatably.

Q: Can you describe a Jenkins pipeline you've worked with?

A: See the full example in Chapter 8 – checkout, build with Maven, test, Sonar Cloud quality check, Docker build, push, then deploy to GKE with a rollout status check.

Q: What is a Jenkinsfile?

A: A file (checked into the repository) that defines the pipeline as code, so the pipeline configuration is version-controlled along with the application code.

Q: What does Maven do in a Java-based build?

A: Maven manages the project's dependencies, compiles the code, runs tests, and packages the application using a standard project structure defined in a `pom.xml` file.

Q: How is a Bitbucket Pipeline different from a Jenkins pipeline?

A: Bitbucket Pipelines is built into Bitbucket itself and configured with a YAML file in the repository, running in Bitbucket's own hosted infrastructure (see Chapter 9). Jenkins is a separate, self-managed automation server, which gives more flexibility but also more setup and maintenance work.

Q: What would you check first if a pipeline suddenly starts failing?

A: Look at the pipeline logs to find which stage failed and the error message, then check whether it's a code issue, a configuration/credentials issue, or an environment issue.

Q: What is a rollback, and how might you handle one?

A: Reverting to a previous, known-good version after a bad deployment – see the automated rollback script in Chapter 11.

17. Monitoring & Logging

Q: What's the difference between Cloud Monitoring and Cloud Logging?

A: Cloud Monitoring tracks metrics over time and can alert you when something crosses a threshold. Cloud Logging collects the actual text output and events from applications and infrastructure, which you search through to understand what happened at a specific point in time.

Q: How would you set up an alert for a production issue?

A: Create an alerting policy on a metric (see the Terraform example in Chapter 10) and configure a notification channel so the team is notified automatically.

Q: If a service is slow, how would you start investigating using these tools?

A: Check Cloud Monitoring dashboards first for obvious resource issues, then check Cloud Logging around the same time window for errors or slow request logs.

Q: Why is centralized logging important in a microservices or multi-service setup?

A: When a request passes through several services, the relevant logs are scattered across each one. Centralized logging pulls them into one place so you can search and correlate events across services.

18. Git & Version Control

Q: What is the difference between `git merge` and `git rebase` ?

A: `git merge` combines two branches and creates a merge commit, preserving full history. `git rebase` replays your commits on top of another branch for a cleaner, linear history, but rewrites commit history.

Q: What is a pull request (or merge request) and why is it useful?

A: A request to merge changes from one branch into another, giving the team a place to review the code, run automated checks, and discuss changes before they're merged.

Q: How do you resolve a merge conflict?

A: Git marks the conflicting sections in the affected files. You manually review both versions, edit the file to remove the conflict markers, then stage and commit the resolved file.

Q: What's a typical branching approach you've used?

A: A main/master branch that always reflects production, feature branches for new work, and pull requests to merge feature branches back in after review and passing CI checks.

19. Linux & Shell Scripting

Q: What are some Linux commands you use regularly, and what do they do?

Command	Purpose
ls -la	List files/directories including hidden ones, with details
cd, pwd	Change directory / print current directory
grep "text" file	Search for a pattern inside a file
tail -f app.log	Watch a log file live as new lines are added
chmod / chown	Change file permissions / ownership
ps aux	List running processes
df -h	Check disk space usage
top / htop	Monitor CPU and memory usage in real time

Q: Why is shell scripting useful in a DevOps role?

A: Shell scripts automate repetitive tasks so they don't need to be typed manually every time, and they can be plugged into pipelines - see the deploy-and-verify script in Chapter 11.

Q: What is a YAML file, and where have you used one?

A: YAML is a human-readable format for structured configuration, using indentation instead of brackets. Used for pipeline configuration (Chapter 9) and Kubernetes manifests (Chapter 7).

20. Agile, Scrum & JIRA

Q: What does a typical sprint look like on your team?

A: A sprint is usually two weeks. It starts with sprint planning, followed by daily stand-ups, and ends with a sprint review/demo and a retrospective.

Q: How do you use JIRA day-to-day?

A: Tracking assigned tickets, updating their status as you work (To Do, In Progress, In Review, Done), logging comments on blockers, and linking tickets to the related pull requests.

Q: What's the difference between a bug, a task, and a story in JIRA?

A: A story is new functionality from the user's perspective. A task is general work that isn't necessarily user-facing. A bug is a defect that isn't working as expected.

Q: How do you handle it when a sprint task takes longer than expected?

A: Flag it early in stand-up rather than waiting until the end of the sprint, explain the cause, and work with the team/lead to decide whether to get help, descope, or carry it into the next sprint.

21. Common HR / Behavioral Questions

Q: Tell me about yourself.

A: Use the 30-second summary from Chapter 2, then add one sentence about what you're looking for next.

Q: Why are you looking to move roles now?

A: Keep it positive and forward-looking – focus on wanting to grow further in a GCP DevOps-focused role, rather than criticizing your current employer or project.

Q: Describe a time you faced a difficult production issue. What did you do?

A: Use the STAR method: Situation, Task, Action (specific steps – checking logs, isolating the cause, applying a fix, ideally referencing the troubleshooting flow in Chapter 22), Result (with a number if possible).

Q: Tell me about a time you disagreed with a teammate or lead. How did you handle it?

A: Pick a real, low-drama example. Show that you explained your reasoning calmly, listened to their side, and reached a resolution together.

Q: How do you keep your skills up to date?

A: Mention specifics: following GCP documentation/release notes, practicing Terraform and Kubernetes in personal projects, and your certifications (GCP Associate Cloud Engineer, HashiCorp Terraform Associate).

Q: Where do you see yourself in a few years?

A: Growing deeper into GCP DevOps/SRE work – more ownership of infrastructure design, deeper Kubernetes and automation skills, and potentially the Professional Cloud DevOps Engineer certification.

Q: Do you have any questions for us?

A: Always have 2–3 ready – for example: "What does the CI/CD and cloud infrastructure setup look like on this team?", "What tools does the team use for monitoring and incident response?", or "What would success look like in this role in the first 6 months?"

22. Scenario-Based Troubleshooting Questions

Q: A deployment just went out and the application is throwing errors. What do you do, step by step?

A: 1) Check Cloud Logging / application logs for the error and stack trace. 2) Check if it correlates with the recent deployment. 3) If it's clearly the new deployment, roll back (see the automated script in Chapter 11). 4) Investigate the root cause in a non-production environment. 5) Communicate status to the team throughout.

Q: A Terraform apply fails halfway through. What's your first move?

A: Read the error message – it usually points to the exact resource and reason. Run `terraform plan` again to see current vs. desired state, fix the underlying issue, and re-apply. Avoid manually editing the state file unless you're confident about what you're doing.

Q: A Pod in GKE is stuck in "CrashLoopBackOff." What would you check?

A: Run `kubect1 describe pod` to check recent events, and `kubect1 logs --previous` to see why the application is exiting – common causes are a misconfiguration, a missing environment variable/secret (see TC-K8S-06 in Chapter 7), or a failed health check.

Q: Your Jenkins build is suddenly taking much longer than usual. How do you investigate?

A: Check if it's a specific stage that's slow by looking at stage timing in the pipeline logs. Consider whether dependencies changed, the build agent is under load, or an external service the pipeline depends on is slow.

Q: You notice a firewall rule that seems overly permissive (open to all IPs). What do you do?

A: Don't change it immediately without understanding why it exists. Then work to narrow the rule to only the specific IP ranges or services that actually need access (compare against the tiered approach in Chapter 4), and test before rolling the change out broadly.

23. Quick Command Cheat Sheet

gcloud (GCP CLI)

Command	Purpose
gcloud config list	Show current project/account configuration
gcloud compute instances list	List Compute Engine VMs
gcloud projects list	List accessible GCP projects
gcloud iam service-accounts list	List service accounts in a project
gcloud container clusters get-credentials NAME	Connect kubectl to a GKE cluster

Docker

Command	Purpose
docker build -t app:v1 .	Build an image from a Dockerfile
docker run -p 8080:8080 app:v1	Run a container, mapping a port
docker ps	List running containers
docker logs <container>	View container logs
docker exec -it <container> sh	Open a shell inside a running container

Kubernetes (kubectl)

Command	Purpose
kubectl get pods	List pods in the current namespace
kubectl describe pod NAME	Show detailed status and events for a pod
kubectl logs NAME	View a pod's logs
kubectl apply -f file.yaml	Create/update resources from a manifest
kubectl rollout status deployment/NAME	Wait and confirm a rollout finished successfully
kubectl get svc / kubectl get deployments	List services / deployments

Terraform

Command	Purpose
terraform init	Initialize working directory and providers
terraform plan	Preview changes before applying
terraform apply	Apply changes to reach desired state
terraform destroy	Remove managed resources
terraform fmt / validate	Format code / check syntax validity

Git

Command	Purpose
git status	Show changed/staged files
git checkout -b feature/x	Create and switch to a new branch
git add . / git commit -m "msg"	Stage and commit changes
git pull / git push	Sync with the remote repository
git log --oneline	View compact commit history

24. Final Tips for Interview Day

- **Know your resume cold.** Every bullet point should have a 20–30 second story behind it, using the talking points in Chapter 2.
- **Practice drawing the four diagrams from Chapter 3** from memory, even roughly – being able to sketch the network layout or CI/CD flow on a whiteboard is a strong signal of real understanding.
- **Walk through at least 2–3 of the Part 2 scenarios out loud**, code and all, until you can explain each line without reading it verbatim.
- **It's okay to say "I've mostly worked with X at a working-knowledge level."** This is more credible than pretending deep expertise you don't have yet.
- **Think out loud during technical questions.** Interviewers often care as much about your problem-solving approach as the final answer.
- **Use real numbers where you have them** – the ~20% performance improvement and ~30% deployment time reduction from your resume are good, specific details to bring up naturally.
- **Prepare 2–3 questions to ask the interviewer** (see Chapter 21).
- **Do one full mock run-through the night before** – read every question in this handbook out loud and answer without looking, then check yourself against the written answer.

All the best, Yamuna! You already have real, hands-on experience with the core tools GCP DevOps roles look for – Terraform, Jenkins, GCP networking, and CI/CD automation. This handbook is here to help you organize and present that experience clearly, back it up with working code, and defend it with test cases – confidently.